HARNESS LAB · MÓDULO 3 · AULA 3.5

Contexto *na prática*

O porquê já veio na 1.2 e na 1.3. Aqui é só o como: o que entra, o que sai e o que precisa morar em arquivo.

`/context` `/clear` `/compact <foco>` `/rewind` `--continue` `/resume` `PLANO.md` `SessionStart compact` `statusLine`

VOCÊ SAI COM

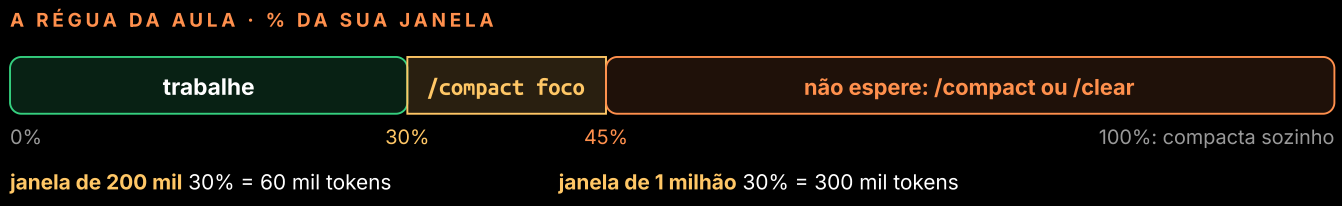
A régua de 30% à vista na status line, o `PLANO.md` da sua feature-alvo e um hook que devolve esse plano ao modelo depois de toda compactação. Sessão curta, estado no disco.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

O problema não é acabar o contexto. É encher. Você quase nunca bate no teto, e perto dele o Claude Code compacta sozinho. Mas compactar sozinho quer dizer que você já se ferrou: o desempenho cai, a qualidade cai, as regras perdem peso. Caber não é funcionar bem. O trabalho é manter na janela só o que a tarefa de agora precisa.

A régua da aula: **limite em 30% da janela, no máximo uns 45%**. E é 30% da *sua* janela. Para ver onde você está, o `/context` da 1.2: arquivos de memória, ferramentas de MCP que dá para desligar, skills empilhadas e o histórico de mensagens.



Depois, os comandos do dia a dia. **Tarefa nova:** `/clear` antes de começar; não comece uma tarefa em cima da outra. **Mesma tarefa, histórico grande:** `/compact` com o foco escrito, para levar só o que precisa ficar. **Foi por um caminho errado:** `/rewind` volta a conversa para antes do primeiro erro, aquele que acumula alucinação, sem precisar de um `/clear`. **Cansou, vai dormir:** amanhã, `--continue` ou `/resume` trazem a última conversa. E se a sessão já acumulou dois erros, a melhor coisa é `/clear`.

> *"After two failed corrections run /clear and write a better prompt instead of a third."*
>
> Documentação oficial do Claude Code · Best practices

O que sobrevive ao `/compact`: o CLAUDE.md da raiz, sempre, com `/compact` ou `/clear` (mantenha o CLAUDE.md só na raiz), e parte das skills já usadas. A conversa vira resumo. O que precisa durar mora em arquivo: decisão que vale sempre no CLAUDE.md; plano e progresso no `PLANO.md`, que mostra onde parou se acabar a luz (ele volta nos próximos módulos); critério de pronto; e o commit, que é histórico.

Sessão curta

Começa limpa e salva em disco o que está sendo feito. Busca e log, subagente. Mudou de assunto, sessão nova.

Usa e abusa do compact

Passou dos 30% no mesmo assunto? `/compact`. Melhor que deixar o contexto crescer.

2 Indo *além da aula*

ALÉM DA AULA · FAÇA A CONTA COM A JANELA QUE VOCÊ TEM DE VERDADE

O tamanho aparece no `/context` e no JSON da status line (`context_window_size`). Pela documentação, Fable, Sonnet 5 em diante, Haiku 5.5 e Opus 4.7 em diante rodam com 1 milhão; Opus 4.6 e Sonnet 4.6, só na variante `[1m]`; `CLAUDE_CODE_DISABLE_1M_CONTEXT=1` volta para 200 mil.

Fontes: Model configuration, "Extended context"; Status line, "Available data".

ALÉM DA AULA · A REDE DE SEGURANÇA PODE FICAR MAIS PERTO

Sem ajuste, a compactação automática roda perto do limite. `/autocompact 400k` faz ela rodar aos 400 mil no modelo atual (`auto` volta ao padrão). Mas ela escolhe sozinha o que manter. Para um foco fixo, a documentação mostra a seção `# Compact instructions` no CLAUDE.md (seção 4).

Fontes: Model configuration, "Set the auto-compact window"; Costs, "Manage context proactively".

ALÉM DA AULA · O /REWIND NÃO DESFAZ TUDO

`Esc` duas vezes, com o campo vazio, abre o mesmo menu: restaurar código e conversa, só um dos dois, ou resumir a partir de um ponto (**Summarize from here** e **up to here**, um `/compact` de um trecho). Arquivo que só comando de terminal mudou não volta (`migrate`, `npm install`, `sed -i`), nem edição de subagente fora dos arquivos da conversa: para isso, o git. Voltar reaproveita o cache; compactar monta um prefixo novo.

Fontes: Checkpointing, "Limitations"; Prompt caching.

ALÉM DA AULA · RETOMAR: O /CLEAR NÃO APAGA, E SESSÃO VELHA CUSTA

A conversa limpa fica salva: `/clear nome` dá nome a ela no `/resume`. O `claude --continue` abre a mais recente *da pasta atual*; `claude --resume` abre a lista. Ao retomar, a conversa inteira vai de novo, e depois de uma noite o cache (5 minutos ou 1 hora) expirou: a primeira volta reprocessa tudo, e um `/compact` em seguida é o mais caro. Sessão curta, `--continue`. Acima dos 30%: commit e `PLANO.md` antes de parar, sessão nova amanhã.

Fontes: Commands, "/clear"; Sessions, "Resume a session"; Prompt caching, "Resuming a session".

ALÉM DA AULA · O QUE MAIS VOLTA DEPOIS DO COMPACT, E O QUE NÃO VOLTA

Voltam do disco: rules sem `paths:`, auto memory e o plano do plan mode; até cinco arquivos lidos ou editados são relidos. Rules com `paths:` e CLAUDE.md de subpasta são resumidos com a conversa e só voltam quando o Claude mexe de novo num arquivo que os aciona. Regra que precisa durar perde o `paths:`. O hook deste material usa o `SessionStart` com matcher `compact`, que roda depois de compactar.

Fontes: Context window, "What survives compaction"; Hooks reference, "SessionStart".

3 Faça *no seu repositório*

Duas peças. No projeto: o hook `.claude/hooks/plano-depois-do-compact.sh` e o `SessionStart` no `.claude/settings.json`, que vão para o git e valem para o time. Só para você: a status line de contexto no seu `~/.claude/`, porque é preferência pessoal. Nada seu é apagado.

1 Confira o jq.

O hook e a status line leem o JSON do Claude Code com ele. Tem que responder com a versão:

Terminal · na raiz do seu projeto — shell

```
jq --version
```

Não tem? Instale com o gerenciador do seu sistema:

Ubuntu e WSL — shell

```
sudo apt install -y jq
```

macOS — shell

```
brew install jq
```

2 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

3 Instale o hook da aula no projeto.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 3.5
```

A saída mostra `criado` para o hook e `criado` ou `somado` para o `settings.json`. Os hooks e as permissões que você já tinha continuam lá; a cópia do original fica em `settings.json.antes-harness-lab`. Ela é só para comparar: apague, ou ponha no `.gitignore`, antes de commitar.

4 Confira o que entrou.

Terminal · na raiz do seu projeto — shell

```
ls -l .claude/hooks/plano-depois-do-compact.sh
jq '.hooks.SessionStart' .claude/settings.json
```

O script precisa do `x` de executável, e a lista precisa ter um item com `"matcher": "compact"`.

5 Crie o PLANO.md da sua feature-alvo.

Troque `frete` pelo nome da sua feature e reescreva o conteúdo do modelo com a sua lista. O primeiro `- [ ]` do arquivo é o próximo item.

Terminal · na raiz do seu projeto — shell

```
A=~/harness-lab-materiais/modulo-3-regra/aula-3.5-contexto-na-pratica
mkdir -p features/frete
cp "$A/modelos/PLANO.md" features/frete/PLANO.md
```

6 Teste o hook fora do Claude Code.

Mande o JSON que o Claude Code manda depois de compactar:

Terminal · na raiz do seu projeto — shell

```
H=.claude/hooks/plano-depois-do-compact.sh
echo '{"source":"compact"}' | "$H" | jq -r .systemMessage
```

o que aparece — com o modelos/PLANO.md

```
plano reinjetado depois do compact: features/frete/PLANO.md,
próximo: frete grátis acima de R$ 300
```

Troque `.systemMessage` por `.hookSpecificOutput.additionalContext` para ver o que o modelo recebe: o plano, o próximo item, as linhas de "Não deu certo" e o lembrete de commit. Sem plano com item pendente, não aparece nada: o hook sai calado, sem sujar o contexto.

7 Ligue a status line de contexto (só para você).

O script vai para `~/.claude/` e o `statusLine` entra no seu `~/.claude/settings.json`. Se você já tem uma status line, nada é trocado (para trocar, rode de novo com `--trocar`).

Terminal · qualquer pasta — shell

```
A=~/harness-lab-materiais/modulo-3-regra/aula-3.5-contexto-na-pratica
"$A/modelos/ligar-statusline.sh"
```

Teste com o JSON que o Claude Code manda para a barra:

Terminal · qualquer pasta — shell

```
J='{"context_window":{"context_window_size":200000,"used_percentage":34}}'
echo "$J" | ~/.claude/statusline-contexto.sh
```

o que aparece — amarelo: passou de 30%

```
Claude · ctx 34% de 200k (68k) · /compact com foco
```

Até 30% sai verde com `ok`; a partir de 45%, vermelho com `/compact ou /clear agora`. Mude a régua com `CTX_AVISO` e `CTX_LIMITE`.

Prefere pedir em vez de instalar? O `/statusline` gera uma para você:

Dentro do Claude Code — sessão

```
/statusline mostre o % de contexto: amarelo de 30% e vermelho de 45%
```

8 **Abra o Claude Code e faça a demo da aula.**

Aceite a confiança na pasta: antes disso, nem o hook nem a status line rodam. O `/hooks` deve listar `SessionStart`. Numa sessão com histórico, olhe o `/context`, compacte com foco e olhe de novo:

Dentro do Claude Code — sessão

```
/hooks
/context
/compact mantenha as decisões do frete e os comandos de teste
/context
> Sem ler arquivo: qual o próximo item do plano e o que não deu certo?
```

Depois do `/compact` aparece `SessionStart:compact says: plano reinjetado depois do compact: ...`. Testado para este material: o modelo respondeu o próximo item e o que não deu certo sem abrir o arquivo. A barra mostra `ctx --` até a primeira resposta depois de compactar: ainda não há número para medir.

9 **Um dia inteiro: uma sessão por tarefa.**

Começo:

Dentro do Claude Code — sessão

```
/clear
> Leia features/frete/PLANO.md e faça o próximo item pendente. Só ele.
```

> **O PEDIDO DO FIM**
>
> "Terminou o item. Faça o commit, atualize o PLANO.md (feito, próximo, o que não deu certo) e me diga em 3 linhas." Depois, `/clear`. Se um erro apareceu de novo, ele vira linha no CLAUDE.md (aula 3.1).

Errou o caminho no meio, ou a sessão de ontem ficou curta? Os outros movimentos:

Dentro do Claude Code — sessão

```
/rewind
/clear frete-faixa-pesado
/resume
```

Terminal · na raiz do projeto, no dia seguinte — claude

```
claude --continue
# para escolher a sessão na lista: claude --resume
```

10 **Opcional: compactação automática mais cedo.**

Em janela de 1 milhão, 400 mil fica perto do teto da régua (45%). Vale para o modelo atual, gravado no seu settings de usuário. Em janela de 200 mil não muda nada: o valor é limitado à janela.

Dentro do Claude Code — sessão

```
/autocompact 400k
```

O que o instalador colocou

.claude/settings.json — instalado pelo instalar.sh · não copie do PDF

```
{
  "hooks": {
    "SessionStart": [{
      "matcher": "compact",
      "hooks": [{ "type": "command", "timeout": 10,
        "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/plano-depois-do-compact.sh" }]
    }]
  }
}
```

O matcher `compact` pega o `/compact` e a compactação automática. Os outros valores são `startup`, `resume`, `clear` e `fork`: o hook não entra neles de propósito, porque a sessão limpa da rotina é você quem aponta para o plano. O script procura em `features/*/PLANO.md` e no `PLANO.md` da raiz (mude com `PLANO_GLOB`), escolhe o plano com item pendente mexido por último, cita os outros e sai sempre com `exit 0`: problema vira saída vazia, nunca bloqueio. O script inteiro, comentado, está em `projeto/.claude/hooks/` no repositório.

4 Arquivos que *reforçam a aula*

`.claude/hooks/plano-depois-do-compact.sh`

"O que você disse na conversa vira resumo." O hook faz o estado do disco voltar sozinho depois de cada compactação, inclusive a automática que você não viu acontecer.

No repositório: modulo-3-regra/aula-3.5-contexto-na-pratica/projeto/.claude/hooks/

`modelos/PLANO.md`

O arquivo que mostra onde parou se acabar a luz: feito, próximo, o que não deu certo e as decisões. É reaproveitado no Módulo 4 (critério de aceite) e no 5 (tarefas do loop).

No repositório: modulo-3-regra/aula-3.5-contexto-na-pratica/modelos/

`modelos/statusline-contexto.sh`

A régua de 30% e 45% sempre na tela, sem precisar rodar o `/context`. Ligada pelo `ligar-statusline.sh`, que não troca uma status line que você já tem.

No repositório: modulo-3-regra/aula-3.5-contexto-na-pratica/modelos/

`modelos/claude-md-contexto.md`

Dois trechos para o CLAUDE.md da raiz, que sobrevive a todo compact: "Onde procurar" (o nome do plano é sempre o mesmo) e "Compact instructions" (o foco que você não digita toda vez).

No repositório: modulo-3-regra/aula-3.5-contexto-na-pratica/modelos/

`modelos/rotina-da-sessao.md`

A rotina da aula numa folha: pedido de começo e de fim, focos de `/compact` que funcionam e qual comando usar em cada situação.

No repositório: modulo-3-regra/aula-3.5-contexto-na-pratica/modelos/

`modelos/claude-md-contexto.md · o que colar no CLAUDE.md` — adapte os caminhos

```
## Onde procurar
- Plano e progresso de cada feature: features/<feature>/PLANO.md.
  O primeiro - [ ] é o próximo item. Ao terminar um item: commit e
  atualize o plano (feito, próximo, o que não deu certo).
- Critério de pronto: features/<feature>/SPEC.md.

# Compact instructions
Ao compactar, mantenha: a feature em andamento e o caminho do PLANO.md
dela, as decisões tomadas nesta sessão, os arquivos alterados e os
comandos de teste exatos. Pode descartar: resultado de busca, log e
conteúdo de arquivo que dá para ler de novo.
```

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
Status line em branco	Pasta sem confiança aceita: a barra segue a regra dos hooks	Reabra o Claude Code e aceite a confiança. `claude --debug` mostra `workspace trust not accepted`
Barra com `ctx --`	Começo da sessão ou logo depois do `/compact`	Normal. O número volta na próxima resposta
A status line da aula não aparece num projeto	O `statusLine` do projeto vence o do usuário	Tire o `statusLine` do projeto, ou aponte o dele para `~/.claude/statusline-contexto.sh`
Depois do `/compact`, nada de `SessionStart:compact says`	Nenhum plano com `- [ ]`, ou plano fora de `features/*/` e da raiz	Rode o passo 6. Plano em outro lugar: `PLANO_GLOB` na frente do `command` no `settings.json`
A sessão nova tenta de novo o que já falhou	"Não deu certo" vazio: a tentativa ficou na conversa	Peça a atualização do plano no fim de todo item (passo 9), com o que não deu e por quê
`/rewind` não desfez a migration ou o arquivo gerado	Mudança feita por comando de terminal não entra no checkpoint	Desfaça pelo git (`git status`, `git restore`) e, no banco, com o rollback da sua stack
`--continue` abriu a conversa errada	Ele abre a mais recente da pasta atual	Rode na raiz do projeto, ou use `claude --resume` e escolha

6 Você sai *com*

- [ ] O `/context` de hoje comparado com o da 1.2, e a janela do seu modelo na ponta da língua (quanto é 30% dela).
- [ ] A status line de contexto na tela, verde até 30%, amarela até 45%.
- [ ] `features/<sua-feature>/PLANO.md` com feito, próximo e não deu certo, versionado no git.
- [ ] `.claude/hooks/plano-depois-do-compact.sh` ligado no `SessionStart` compact, e a mensagem dele aparecendo depois de um `/compact`.
- [ ] Um dia de uso com uma sessão por tarefa: começo limpo, commit e plano atualizado no fim.

PROMETIDO NA AULA

- ✓ "Lembrando lá do comando /context": o que olhar nele está na seção 1, e a demo com `/context` antes e depois do `/compact` na seção 3, passo 8.
- ✓ "Coloca o seu limite como 30% e aceita no máximo uns 45%": a status line pinta essa régua (seção 3, passo 7, e `modelos/statusline-contexto.sh`).
- ✓ `/clear`, `/compact` com foco, `/rewind`, `--continue` e `/resume`: passos 8 e 9, com a tabela de quando usar cada um em `modelos/rotina-da-sessao.md`.
- ✓ "O PLANO.md vai te mostrar onde parou": `modelos/PLANO.md` (passo 5) e o hook que devolve o plano depois de compactar (passo 6).
- ✓ "Experimenta usar o compact no seu dia a dia": os focos de `/compact` em `modelos/rotina-da-sessao.md` e o `/autocompact` no passo 10.

7 *Glossário*

Janela de contexto	Tudo que o modelo enxerga na sessão: instruções, arquivos lidos, respostas. 200 mil ou 1 milhão de tokens, conforme o modelo.
Compactação	Troca o histórico da conversa por um resumo. Manual com `/compact`, automática perto do limite.
Foco do compact	O texto depois de `/compact`: diz o que o resumo não pode perder.
Checkpoint	O estado dos arquivos antes de cada mensagem sua. É para onde o `/rewind` volta.
Sessão	Uma conversa salva em disco. `/clear` abre outra; `--continue` e `/resume` voltam a uma antiga.
Estado no disco	O que precisa durar mais que a sessão: CLAUDE.md, `PLANO.md`, spec e commit.
SessionStart	Evento de hook no começo da sessão. Com matcher `compact`, roda logo depois de compactar.
Status line	A barra embaixo do campo de digitação. Roda um script com o JSON da sessão.

8 Para *ir além*

- Explore the context window: a tabela "What survives compaction", mecanismo por mecanismo.
- Checkpointing: as opções do `/rewind`, os resumos parciais e o que ele não desfaz.
- Sessions: `--continue`, `--resume`, nomear, ramificar e onde ficam as conversas.
- Status line: todos os campos do JSON, inclusive `context_window` e uso do limite do plano.
- Prompt caching: por que retomar sessão velha e compactar no meio da tarefa custam mais.

Próxima: *Aula 4.1 · Quem decide se está pronto não é o modelo*

Módulo 3 fechado: CLAUDE.md enxuto, guidelines por versão, a primeira skill, o test-runner, os subagentes e sessão curta com estado no disco. O agente segue o padrão do projeto. Agora, o Módulo 4: o modelo foi treinado para dar uma resposta plausível, não uma resposta certa. Pronto é o que passa no teste.